Documentación del Proyecto

Tienda Online con Integración de API | Manual Técnico y de Arquitectura

1. ¿Qué es una API y cómo funciona?

> **La Analogía del Restaurante**
> Imagina que vas a un restaurante. Tú eres el cliente (el frontend o tu página web), la cocina es el servidor (la base de datos) y el mesero es la **API**. Tú no vas a la cocina a buscar los ingredientes; le entregas tu orden al mesero, él la lleva a la cocina y regresa a tu mesa con la comida servida. En nuestro código, pedimos productos y la API nos trae los datos.

El rol de Fake Store API

Fake Store API es un servidor público diseñado específicamente para practicar desarrollo web. Contiene una base de datos ficticia de productos de comercio electrónico. En lugar de crear una base de datos desde cero, tu código JavaScript simplemente le pide la información a esta API a través de internet para construir el catálogo visual.

Los Endpoints (Rutas de acceso)

Para pedir información específica, la API proporciona diferentes URLs o rutas, conocidas como *endpoints*. En tu tienda, utilizas dos principales:

- `GET /products`: Al consultar esta ruta, el servidor devuelve un listado completo (un arreglo) con todos los productos disponibles.
- `GET /products/{id}`: Al consultar esta ruta reemplazando el {id} por un número (ejemplo: `/products/1`), la API filtra su base de datos y te devuelve únicamente la información detallada de ese producto exacto.

2. Flujo de Datos y Formato

El formato de intercambio (JSON)

Cuando la API responde a tu petición, no envía páginas web, colores ni diseños; envía datos puros en un formato llamado **JSON** (JavaScript Object Notation). Es básicamente texto estructurado que JavaScript entiende de forma nativa. Tu código utiliza la instrucción `.json()` para convertir ese texto en objetos reales de JavaScript, lo que te permite extraer el título, precio e imagen.

El ciclo de la petición (Fetch)

Tu archivo de JavaScript utiliza la función `fetch()` para ir a buscar esta información. Dado que la petición viaja por internet y puede tardar dependiendo de la conexión, se utilizan funciones asíncronas (`async / await`). Esto le dice a tu página: *"ve a buscar los datos y espera pacientemente a que lleguen antes de intentar dibujarlos en la pantalla"*.

3. Estructura del Proyecto

Tu proyecto está organizado en tres partes principales, representando los pilares del desarrollo web:

```
src/ ├── index.html # EL ESQUELETO: Define dónde van los elementos. ├── css/ | └── api-store.css # EL MAQUILLAJE: Da tamaños y arregla imágenes. └── js/ └── api-store.js # EL CEREBRO: Se conecta a la API y dibuja HTML.
```

El archivo HTML contiene contenedores vacíos (con IDs específicos como `id="catalogo"`). El archivo JavaScript se encarga de ir a buscar los datos a la API y luego "inyectar" código HTML lleno de información dentro de esos contenedores vacíos.

4. Explicación del Código (Parte 1)

Función 1: obtenerProductos()

Esta es la función más importante. Es la que ejecuta la llamada al "mesero" (la API). Utiliza un bloque `try-catch` para manejar posibles caídas de internet.

```
async function obtenerProductos() { try { // 'await' significa: Espera aquí hasta que la API responda const response = await fetch('https://fakestoreapi.com/products'); if (!response.ok) { throw new Error('Error al obtener los productos'); } // Convierte el texto JSON a una lista de objetos JavaScript return await response.json(); } catch (error) { console.error('Error:', error); return []; // Retorna lista vacía para evitar errores graves } }
```

Función 2: mostrarCatalogo(productos)

Esta función recibe la lista de productos que trajo la API. Busca el contenedor en el HTML y utiliza el método `.map()` para generar una tarjeta visual para cada elemento de la lista.

```
function mostrarCatalogo(productos) { const contenedor = document.getElementById('catalogo'); // Recorre cada producto y genera HTML inyectando sus variables contenedor.innerHTML = productos.map(producto => ` <div class="col-12 col-md-4"> <div class="card"> <img src="${producto.image}" alt="${producto.title}"> <p>$${producto.price}</p> <button onclick="mostrarDetalles(${producto.id})">Ver</button> </div> </div> `).join(''); }
```

5. Explicación del Código (Parte 2)

Función 3: mostrarOfertas(productos)

Hace un trabajo idéntico al catálogo, pero antes de dibujar el HTML en pantalla, filtra la lista para quedarse únicamente con tres productos específicos (posiciones 0, 3 y 6 del arreglo).

```
function mostrarOfertas(productos) { const contenedor = document.getElementById('ofertas'); // Las listas en programación empiezan a contar desde cero // Extraemos el 1ro, el 4to y el 7mo producto const ofertas = [productos[0], productos[3], productos[6]].filter(Boolean); // Luego hace lo mismo que el catálogo: map() para crear el HTML... contenedor.innerHTML = ofertas.map(producto => `...`).join(''); }
```

Función 4: mostrarCarrusel(productos)

Toma la lista completa de productos proveniente de la API, la desordena matemáticamente al azar y selecciona los 3 primeros para inyectarlos en la estructura de carrusel de Bootstrap.

```
function mostrarCarrusel(productos) { const contenedor = document.getElementById('carrusel'); // Truco JS: Desordena la lista (sort + random) y corta 3 (slice) const productosAleatorios = [...productos] .sort(() => 0.5 - Math.random()) .slice(0, 3); // Construye los indicadores y los items del carrusel // ... }
```

6. Interacción y Puesta en Marcha

Función 5: mostrarDetalles(idProducto)

Cuando el usuario hace clic en "Ver Detalles" en cualquier tarjeta, esta función recibe el ID de ese producto específico y hace una **nueva petición a la API** enfocada solo en ese artículo.

```
async function mostrarDetalles(idProducto) { // Petición al endpoint específico del producto const response = await fetch(`https://fakestoreapi.com/products/${idProducto}`); const producto = await response.json(); // Inyecta datos en el modal (ventana emergente) const modalContent = document.querySelector('#productoModal .modal-content'); modalContent.innerHTML = `...HTML con ${producto.description}...`; // Activa el modal de Bootstrap visualmente const modal = new bootstrap.Modal(document.getElementById('productoModal')); modal.show(); }
```

Función 6: inicializar()

Es el director de la orquesta. Sin esta función, las demás nunca harían su trabajo. Se ejecuta automáticamente cuando la página web termina de cargar su esqueleto HTML.

```
async function inicializar() { // 1. Obtiene TODOS los productos de internet const productos = await obtenerProductos(); if (productos.length > 0) { // 2. Reparte la información a las 3 secciones visuales mostrarCarrusel(productos); mostrarOfertas(productos); mostrarCatalogo(productos); } } // "Cuando cargues el HTML, ejecuta inicializar()" document.addEventListener('DOMContentLoaded', inicializar);
```

Resumen de tu Aprendizaje
Has logrado crear una aplicación web dinámica. Ya no escribes a mano cada producto en el HTML; ahora tienes un código inteligente que va a una base de datos externa, extrae la información en tiempo real y construye la interfaz visual automáticamente. ¡Esta es la arquitectura exacta de plataformas como Amazon o Netflix!